Security findings report

ms-adithya · valorant-challenge-tracker

Organization	ms-adithya
Repositories	valorant-challenge-tracker
Scan data	Latest completed scan per repository
Generated	6 Oct 2026, 4:48 pm
Filters	State: Open

Findings by severity

Critical	High	Medium	Low
0	4	10	4

18 findings, one page per finding.

FINDING 1 OF 18

XSS

High Open CWE-79 · Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

Repository	ms-adithya/valorant-challenge-tracker
Location	`js/analytics-panels.js:20-20`
Detected	5 Oct 2026, 5:59 pm

DESCRIPTION

Analytics panels insert imported agent and map strings directly into HTML text and quoted attributes. A malicious label can therefore create elements and event handlers that execute in the application origin, rather than remaining display data.

Analytics panels insert imported agent and map names directly into HTML. An attacker-supplied match or backup can therefore execute JavaScript in the recipient’s application origin, gaining access to local tracker data and any active authenticated application capabilities.

Evidence

• Source/control: Agent and map strings from an attacker-supplied backup that the victim restores, or ready match rows that the victim imports.

• Sink/decision: Unescaped innerHTML interpolation in renderTopAgents at js/analytics-panels.js:20, renderDist at line 35, renderWinBars at line 41, and the top-map renderer at index.html:566-569.

• Missing mitigation: The final analytics HTML consumers do not encode imported labels for their HTML text and attribute contexts. Input normalization preserves malicious markup instead of preventing it from becoming executable content.

SUGGESTED FIX

Build these analytics labels with DOM nodes and textContent, and assign tooltip text through DOM properties. If HTML templates remain, encode every agent and map value for its insertion context in all affected analytics renderers.

Build the analytics elements with DOM APIs and assign names through textContent. If HTML templates remain, encode every imported value for its HTML text or attribute context across all affected renderers.

FINDING 2 OF 18

XSS

High Open CWE-79 · Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

Repository	ms-adithya/valorant-challenge-tracker
Location	`js/analytics-panels.js:20-20`
Detected	5 Oct 2026, 5:59 pm

DESCRIPTION

The top-agents table renders imported agent and best-map names as HTML without escaping them. A crafted match file can cause script execution in the application origin and allow access to the victim's application data.

The top-agents renderer inserts stored agent names and best-map names directly into HTML. An attacker-supplied backup can restore those strings without passing through match preview, causing script execution when the tracker renders the restored records.

Evidence

• Source/control: Agent and map fields in user-selected match import files.

• Sink/decision: The x.agent and b[0] interpolations in topAgentsTable.innerHTML.

• Missing mitigation: The table lacks text-only rendering or HTML escaping for imported agent and map names.

SUGGESTED FIX

Render agent names, portrait initials and best-map names using textContent, or escape each string before HTML interpolation. Numeric aggregation does not sanitize the associated names.

Build agent and best-map labels with textContent, or HTML-encode them at the rendering boundary. Apply the same output protection to the other analytics renderers that consume stored agent and map names.

FINDING 3 OF 18

XSS

High Open CWE-79 · Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

Repository	ms-adithya/valorant-challenge-tracker
Location	`js/challenge-archive.js:17-18`
Detected	5 Oct 2026, 5:59 pm

DESCRIPTION

Imported challenge IDs are escaped as JavaScript strings and then inserted into HTML event-handler attributes without complete HTML encoding. HTML parsing can decode attacker-supplied character references into JavaScript delimiters, allowing a challenge action click to execute injected statements.

Evidence

• Source/control: A nonempty challenge ID in a user-selected attacker-authored backup.

• Sink/decision: Inline onclick attributes constructed by renderArchive, renderArchiveBrowser, and renderSetupRestore.

• Missing mitigation: The complete event-handler attribute is not encoded for HTML after JavaScript string escaping. Supplied identifiers therefore can change executable handler code instead of remaining data.

SUGGESTED FIX

Replace generated inline handlers with addEventListener and pass IDs through safely assigned DOM properties or datasets. This removes the need to compose JavaScript and HTML escaping for imported identifiers.

FINDING 4 OF 18

XSS

High Open CWE-79 · Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

Repository	ms-adithya/valorant-challenge-tracker
Location	`js/import-preview.js:57-57`
Detected	5 Oct 2026, 5:59 pm

DESCRIPTION

The import preview interpolates file-supplied agent, map and rank text into HTML without escaping. Opening a crafted match file can execute JavaScript in the tracker origin before the user commits the import, allowing access to or modification of browser-stored tracker data.

The import preview treats file-supplied labels and retained rank-status text as HTML. Crafted markup can execute an event handler in the tracker origin before import approval, exposing private browser-stored history and permitting changes with the page's authority.

Evidence

• Source/control: Attacker-authored JSON, CSV or TSV selected through the Import matches file input.

• Sink/decision: The importMatchesPreview innerHTML assignment at lines 49–59, specifically the unescaped match fields at line 57; rankStatusAdjusted is also interpolated at line 52.

• Missing mitigation: Imported display text is not escaped before the browser parses the preview as HTML. Rejecting invalid rows when saving does not protect the earlier preview.

SUGGESTED FIX

Render imported fields with textContent, or apply escapeHtml to every dynamic text value interpolated into the preview, including rank-status adjustment text. Keep HTML structure and CSS classes derived from fixed application values.

Construct preview elements with DOM APIs and assign imported values through textContent. If HTML templates remain, encode every dynamic string, including rejected values and original text included in correction messages.

FINDING 5 OF 18

XSS

Medium Open CWE-79 · Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

Repository	ms-adithya/valorant-challenge-tracker
Location	`index.html:568-568`
Detected	5 Oct 2026, 5:59 pm

DESCRIPTION

The top-maps renderer inserts imported map names into HTML without escaping. An attacker-supplied match file can therefore introduce executable event handlers that can read or modify application data in the victim's browser.

Evidence

• Source/control: The map field in a user-selected CSV, TSV or JSON match import, preserved by validateImportedMatch().

• Sink/decision: The z.name interpolation in markup assigned to top-maps-list.innerHTML.

• Missing mitigation: Imported map names are treated as markup at the display boundary. Match validation does not make those strings safe for HTML.

SUGGESTED FIX

Construct the map-name element with textContent, or apply escapeHtml() to z.name before inserting it into HTML. Keep imported names as plain data throughout storage and aggregation.

FINDING 6 OF 18

Denial of service

Medium Open CWE-407

Repository	ms-adithya/valorant-challenge-tracker
Location	`js/analytics-insights.js:60-61`
Detected	5 Oct 2026, 5:59 pm

DESCRIPTION

The matrix constructs every combination of distinct imported agent and map labels and scans the full match collection for each combination. N records with independently distinct labels produce N cubed match inspections and N squared cells synchronously, making a small imported dataset cause disproportionate browser work.

Evidence

• Source/control: Agent and map strings in a victim-selected match import or restored backup.

• Sink/decision: The Cartesian-product loops and per-cell scope.filter at line 60, followed by the expanded markup assignment to innerHTML at line 61.

• Missing mitigation: The renderer has no effective dimension or cell budget before generating the Cartesian product. It also repeatedly scans the full dataset instead of computing pair statistics once.

SUGGESTED FIX

Aggregate observed agent/map pairs in one pass, then enforce a maximum displayed cell count before constructing markup. Paginate or virtualize larger matrices so imported labels cannot force synchronous cross-product expansion.

FINDING 7 OF 18

XSS

Medium Open CWE-79 · Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

Repository	ms-adithya/valorant-challenge-tracker
Location	`js/analytics-panels.js:35-35`
Detected	5 Oct 2026, 5:59 pm

DESCRIPTION

The distribution-card renderer inserts imported agent and map names into HTML without escaping. A malicious name can create executable browser event handlers and read or modify application data.

Evidence

• Source/control: Agent or map strings from user-selected CSV, TSV or JSON match files.

• Sink/decision: The n interpolation in the distribution container's innerHTML assignment.

• Missing mitigation: Grouping match records does not make their names safe for HTML, and the display operation supplies no escaping.

SUGGESTED FIX

Use textContent for distribution names and avatar text, or apply escapeHtml() to each interpolated string. Preserve the fixed agent/map selector as a trusted rendering parameter.

FINDING 8 OF 18

XSS

Medium Open CWE-79 · Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

Repository	ms-adithya/valorant-challenge-tracker
Location	`js/analytics-panels.js:41-41`
Detected	5 Oct 2026, 5:59 pm

DESCRIPTION

The win-rate bars interpolate imported names into an HTML attribute and element body without escaping. Crafted names can introduce event handlers or additional markup that executes script with access to application data.

Evidence

• Source/control: Agent and map fields preserved from user-selected match import files.

• Sink/decision: The title attribute and body containing n in the win-bar container's innerHTML assignment.

• Missing mitigation: The renderer lacks escaping for both HTML text and the quoted title attribute containing imported names.

SUGGESTED FIX

Create the name element with textContent and assign title through its DOM property. If retaining template markup, use escapeHtml() for both occurrences of n.

FINDING 9 OF 18

XSS

Medium Open CWE-79 · Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

Repository	ms-adithya/valorant-challenge-tracker
Location	`js/challenge-archive.js:17-18`
Detected	5 Oct 2026, 5:59 pm

DESCRIPTION

The archive renderer embeds imported challenge IDs inside JavaScript strings in HTML event attributes. Its escape helper leaves HTML character references intact, allowing the browser to decode an encoded apostrophe into a string delimiter and execute attacker-supplied code when an action is clicked.

Evidence

• Source/control: An existing challenge id in an attacker-supplied JSON backup selected through the application's backup restore controls.

• Sink/decision: The safeId interpolations inside inline onclick handlers for challenge actions.

• Missing mitigation: The renderer does not keep imported IDs separate from executable JavaScript, and its helper fails to encode the surrounding HTML attribute context.

SUGGESTED FIX

Replace inline onclick attributes with addEventListener() handlers that receive IDs as data, using closures or DOM data properties. This avoids interpreting imported IDs as code and removes the need for nested HTML and JavaScript escaping.

FINDING 10 OF 18

Denial of service

Medium Open CWE-400 · Uncontrolled Resource Consumption

Repository	ms-adithya/valorant-challenge-tracker
Location	`js/challenge-report.js:110-110`
Detected	5 Oct 2026, 5:59 pm

DESCRIPTION

The PDF generator writes the complete imported challenge name on every page, while imported match labels independently increase the number of pages. These two input dimensions multiply into PDF size and synchronous browser work, allowing a modest backup to make report generation stall the application tab and allocate substantially more memory than the input size.

Evidence

• Source/control: Challenge names and match agent/map labels in user-selected backup JSON.

• Sink/decision: The per-page doc.text(stamp, ...) write, followed by complete PDF Blob serialization.

• Missing mitigation: The footer has no fixed display-length limit, and report generation has no effective text or page budget. Long imported labels can therefore cause the full long name to be emitted repeatedly.

SUGGESTED FIX

Truncate footer names to a fixed display length and validate imported name and label lengths before report generation. Bound pagination during generation so oversized reports stop before repeated footer writes and complete PDF serialization.

FINDING 11 OF 18

Denial of service

Medium Open CWE-400 · Uncontrolled Resource Consumption

Repository	ms-adithya/valorant-challenge-tracker
Location	`js/import-commit.js:19-19`
Detected	5 Oct 2026, 5:59 pm

DESCRIPTION

The importer expands every row across every attacker-supplied header, even when that row contains only one cell. A small sparse file can therefore cause quadratic property allocation and synchronous work, exhausting memory or making the tracker tab unresponsive.

Evidence

• Source/control: A selected CSV or TSV with many distinct headers and many short nonempty rows.

• Sink/decision: Nested row and header mapping constructs full-width objects with Object.fromEntries.

• Missing mitigation: The importer does not bound the product of row count and column count before allocating objects. Unknown headers also consume properties despite not being needed for match import.

SUGGESTED FIX

Enforce column and row limits before object construction and reject inconsistent row widths. Construct objects only from recognized match fields so arbitrary headers cannot multiply allocation.

FINDING 12 OF 18

Denial of service

Medium Open CWE-1333 · Inefficient Regular Expression Complexity ('ReDoS')

Repository	ms-adithya/valorant-challenge-tracker
Location	`js/import-parse.js:35-35`
Detected	5 Oct 2026, 5:59 pm

DESCRIPTION

Score normalization applies an unanchored expression with a greedy digit run to an unrestricted string. A long digit string without a score separator causes repeated suffix scans and quadratic processing before validation, making the tracker tab unresponsive.

Evidence

• Source/control: An imported JSON score property or delimited Score column, with a separate score field absent.

• Sink/decision: String(o.score).match executes the unanchored score expression synchronously.

• Missing mitigation: The importer does not limit score-string length before matching, and the expression can restart its scan at each digit position.

SUGGESTED FIX

Reject oversized score strings before normalization and use a bounded, anchored score grammar or a linear parser.

Avoid searching for a score substring within arbitrary-length text.

FINDING 13 OF 18

XSS

Medium Open CWE-79 · Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

Repository	ms-adithya/valorant-challenge-tracker
Location	`js/navigation.js:4-4`
Detected	5 Oct 2026, 5:59 pm

DESCRIPTION

The archive browser places imported archive IDs inside an inline JavaScript handler with escaping that preserves HTML character references. The browser decodes those references before compiling the handler, allowing a crafted ID to execute JavaScript when Delete is clicked and access or modify tracker data.

Evidence

• Source/control: The id field of an archived challenge in a user-selected backup JSON file.

• Sink/decision: The Delete button’s onclick attribute generated by renderArchiveBrowser.

• Missing mitigation: The renderer does not keep imported IDs separate from executable JavaScript. The current escaping misses the HTML-decoding step that precedes inline-handler compilation.

SUGGESTED FIX

Replace the inline onclick with a safely encoded data attribute and a delegated click listener that passes the ID as data to deleteArchivedChallenge. This removes the need to serialize imported IDs into JavaScript source.

FINDING 14 OF 18

XSS

Medium Open CWE-79 · Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

Repository	ms-adithya/valorant-challenge-tracker
Location	`js/setup-restore.js:19-19`
Detected	5 Oct 2026, 5:59 pm

DESCRIPTION

The empty-state restore panel embeds imported archive IDs in inline JavaScript using an escaping helper that leaves HTML character references intact. Browser decoding can turn a crafted ID into executable handler code, allowing access to or modification of tracker data when Delete is clicked.

Evidence

• Source/control: The id field of an archived challenge in a user-selected backup JSON file.

• Sink/decision: The Delete button’s onclick attribute generated by renderSetupRestore.

• Missing mitigation: Imported IDs remain part of executable handler source, and the escaping does not account for HTML entity decoding before JavaScript compilation.

SUGGESTED FIX

Render the Delete button with an HTML-escaped data attribute and handle clicks through a delegated listener. Pass the retrieved ID directly to deleteArchivedChallenge instead of embedding it in inline JavaScript.

FINDING 15 OF 18

Injection

Low Open CWE-1236 · Improper Neutralization of Formula Elements in a CSV File ('CSV Injection')

Repository	ms-adithya/valorant-challenge-tracker
Location	`js/challenge-report.js:11-11`
Detected	5 Oct 2026, 5:59 pm

DESCRIPTION

CSV serialization preserves spreadsheet formula prefixes in imported text. When a receiving spreadsheet evaluates those cells, attacker-chosen formulas execute instead of displaying the original literal text, potentially changing exported content or disclosing other cells through permitted network functions.

Evidence

• Source/control: Imported match notes, agent names, and map names from selected CSV, TSV, JSON, or backup files.

• Sink/decision: csvCell serialization used by exportMatchesCsv and exportChallengeCSV before CSV download.

• Missing mitigation: Untrusted text cells receive CSV structural escaping without a policy that prevents spreadsheet formula interpretation.

SUGGESTED FIX

Apply a documented formula-neutralization policy to untrusted text fields in both exporters, including relevant prefixes after leading whitespace or control characters. Keep validated numeric fields numeric and verify the policy against the spreadsheet applications the export supports.

FINDING 16 OF 18

Authorization bypass

Low Open

Repository	ms-adithya/valorant-challenge-tracker
Location	`js/cloud/cloud-sync.js:365-365`
Detected	5 Oct 2026, 5:59 pm

DESCRIPTION

Rejecting the transfer prompt leaves anonymous-session records in application state. The destination account's empty initial snapshot retains those records and enables synchronization, allowing a later save to upload them despite the rejection.

Evidence

• Source/control: Valid attacker-created challenges and matches in a shared browser's anonymous session.

• Sink/decision: The empty-snapshot decision enables pushChanges(), which creates retained records beneath the authenticated destination UID using batch.set() at lines 185-193.

• Missing mitigation: The client does not distinguish rejected incoming records from records authorized for synchronization under the destination identity.

SUGGESTED FIX

After successful authentication, initialize destination state separately and retain incoming records only when the transfer was approved. Clear or quarantine rejected records before enabling destination-account writes.

FINDING 17 OF 18

Sensitive data exposure

Low Open CWE-200 · Exposure of Sensitive Information to an Unauthorized Actor

Repository	ms-adithya/valorant-challenge-tracker
Location	`js/cloud/cloud-sync.js:365-365`
Detected	5 Oct 2026, 5:59 pm

DESCRIPTION

Application records are not associated with their owning account during authentication transitions. A surviving tab retains the previous account's records when the replacement account's first snapshot is empty, exposing them after sign-out and allowing subsequent saves under the replacement identity.

Evidence

• Source/control: Previously authenticated challenge and match records already loaded in a surviving browser tab's data, activeChallenges, and archives globals.

• Sink/decision: The empty-snapshot branch preserves account-owned records and enables synchronization. Those records remain readable through the UI and exportBackupJson(), and pushChanges() can create copies beneath the current UID.

• Missing mitigation: Authentication loss and account changes do not clear account-owned memory and views in every tab. Synchronization callbacks also lack a check that their captured account and session generation remain current before applying records.

SUGGESTED FIX

Bind in-memory records and browser caches to their owning UID, and clear previous-account records and views on authentication loss or identity changes in every tab. Invalidate old asynchronous work before starting the next session, and allow empty-snapshot preservation only for records belonging to that session or an approved transfer.

FINDING 18 OF 18

Injection

Low Open CWE-1236 · Improper Neutralization of Formula Elements in a CSV File ('CSV Injection')

Repository	ms-adithya/valorant-challenge-tracker
Location	`js/export.js:16-18`
Detected	5 Oct 2026, 5:59 pm

DESCRIPTION

Imported notes are serialized into CSV without neutralizing spreadsheet formula prefixes. Opening the export in a compatible spreadsheet can evaluate attacker-supplied formulas, including formulas that disclose other exported cells when external requests are permitted.

Evidence

• Source/control: Notes in an attacker-supplied CSV, TSV, or JSON match file selected by the victim.

• Sink/decision: exportMatchesCsv passes stored notes to csvCell and downloads the resulting CSV.

• Missing mitigation: The exporter does not preserve untrusted text as literal text at the spreadsheet interpretation boundary. CSV quotation alone does not provide this protection.

SUGGESTED FIX

Neutralize formula prefixes in exported text cells using a treatment verified against supported spreadsheet applications. Alternatively, export a spreadsheet format that explicitly marks those cells as text.